

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра Н2 Программная инженерия и интеллектуальные системы
шифр наименование кафедры, по которой выполняется работа
Дисциплина Структуры и организация данных
наименование дисциплины

УЧЕБНО-ПРАКТИЧЕСКАЯ РАБОТА
№2

тема «Нелинейные структуры данных»
Вариант 15

ОБУЧАЮЩИЙСЯ

группы Н253Б

Руднев Е. К.

подпись

фамилия и инициалы

дата сдачи

ПРОВЕРИЛ

ст. преподаватель каф. Н2

ученая степень, ученое звание, должность

Наурусова Г.А.

подпись

фамилия и инициалы

Оценка / балльная оценка

дата проверки

Санкт-Петербург
2026 г.

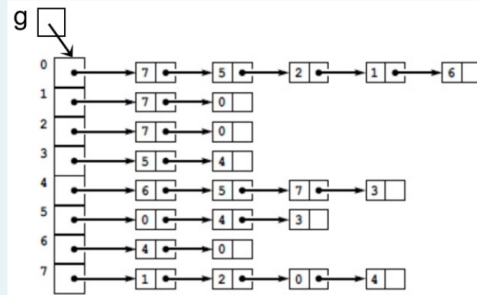
Задание 2.

В некоторой библиотеке есть два класса, в которых определены одинаковые операции для работы с ориентированным взвешенным графом.

В первом классе граф представлен матрицей весов, структура хранения которой – двумерный массив элементов типа unsigned int.

Во втором классе граф представляется списками смежности, структура хранения каждого из которых – односвязный линейный список, значением поля данных каждого элемента является номер вершины и вес дуги. Указатели на головы списков размещаются в массиве. Схематичное изображение структуры хранения в целом приведено на рисунке ниже. Указатель g объявлен так:

```
struct node
{
    short int number_node;
    unsigned int weight;
    node* next;
} **g;
```



Укажите минимальный порядок графа, для хранения которого в объекте второго класса будет выделяться гарантированно меньше памяти, чем в объекте первого класса, если степень любой вершины не будет превышать восемнадцати, sizeof(int) = 4 байта, размер каждого указателя – 8 байт, а поля структуры располагаются вплотную друг к другу.

Решение задачи:

Решение:

- 1) Память для матрицы:
 - Размер: $n \times n$
 - Вес элементов: 4 байта
$$4 \cdot n \cdot n = 4n^2 \text{ байт}$$
- 2) Память для списков смежности:
 1. Размер одного узла:
 - short in number_node = 2 байта
 - unsigned int weight = 4 байта
 - node* next = 8 байт
$$2 + 4 + 8 = 14 \text{ байт}$$
 2. Общая память:
 - Массив указателей g: $n \times 8 \text{ байт}$
 - Узлы списков: $(n \times 18) \times 14 \text{ байт}$
$$8n + 252n = 260n \text{ байт}$$
- 3) Условие задачи: списки занимают меньше памяти

$$260n < 4n^2$$

$$260 < 4n$$

$$n > 65 \Rightarrow \text{минимальное среднее значение } 66$$

Минимальное количество вершин графа, при котором структура хранения в виде списков смежности занимает меньше памяти, чем матрица весов, равно 66.

Ответ: 66

Задание 3. Написать программы для решения задачи.

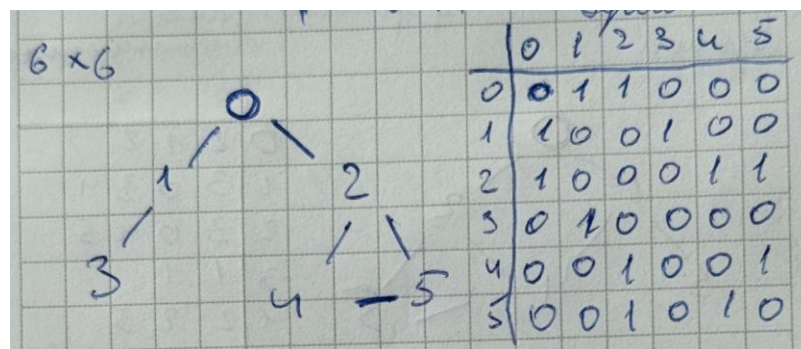
Граф представляется двумя способами (матрицей смежности или весов и списками

смежности). Для каждого представления требуется написать отдельную программу решения задачи, используя алгоритм, наиболее подходящий для используемой структуры хранения. Для тестирования программ требуется создать файлы с описанием графов соответствующим способом (матрицей смежности или весов для программы, обрабатывающей матрицы, списками смежных вершин для программы, обрабатывающей списки смежности). Формат файлов должен быть таким, чтобы чтение из них данных и построение/заполнение структуры хранения требовали минимальных усилий.

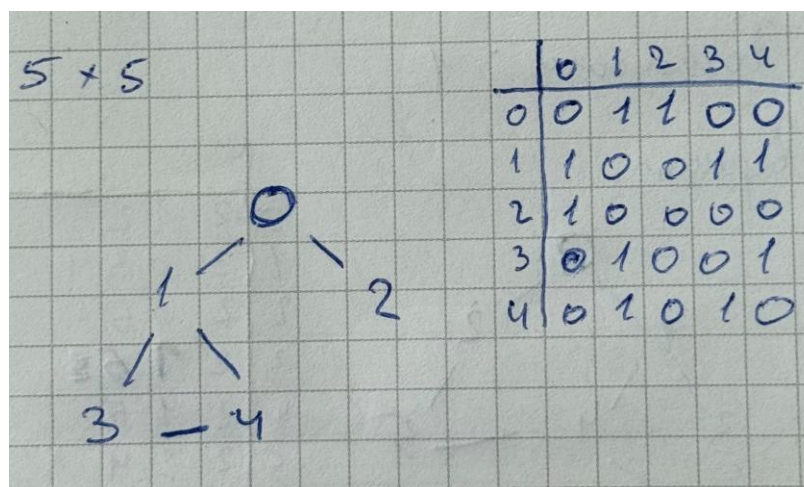
На территории крупного завода организована система электроснабжения, включающая в себя подстанции (одну и более) и цеха – потребители электроэнергии. Ввиду большой территории завода подключить все цеха напрямую к подстанциям невозможно, поэтому некоторые из них запитываются от соседних цехов. Напишите программу для определения, существуют ли в системе такие цеха, выход из строя системы питания которых приведёт к отключению других потребителей энергии.

Тестовый граф:

Матрица смежности (программа 1):

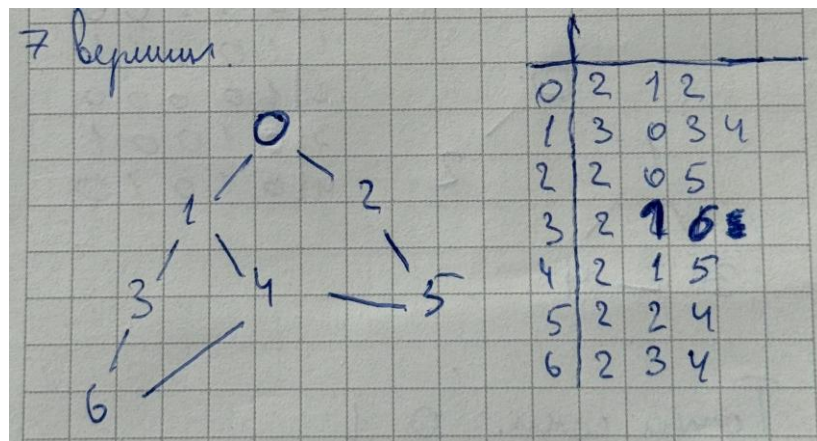


Граф 1



Граф 2

Список смежности (программа 2):



Граф 3

Ожидаемый результат:

Граф 1: при удалении вершин 0, 1 и 2 целостность графа нарушается. Эти вершины являются точками сочленения.

Граф 2: при удалении вершин 0 и 1 целостность графа нарушается. Эти вершины являются точками сочленения.

Граф 3: при удалении любых вершин целостность графа не нарушается. Никакая из вершин не является точкой сочленения.

Структура хранения для представления графа матрицей смежности:

i \ j →	0	1	2	3	4	5
0	0	1	1	0	0	0
1	1	0	0	1	0	0
2	1	0	0	0	1	1
3	0	1	0	0	0	0
4	0	0	1	0	0	1
5	0	0	1	0	1	0

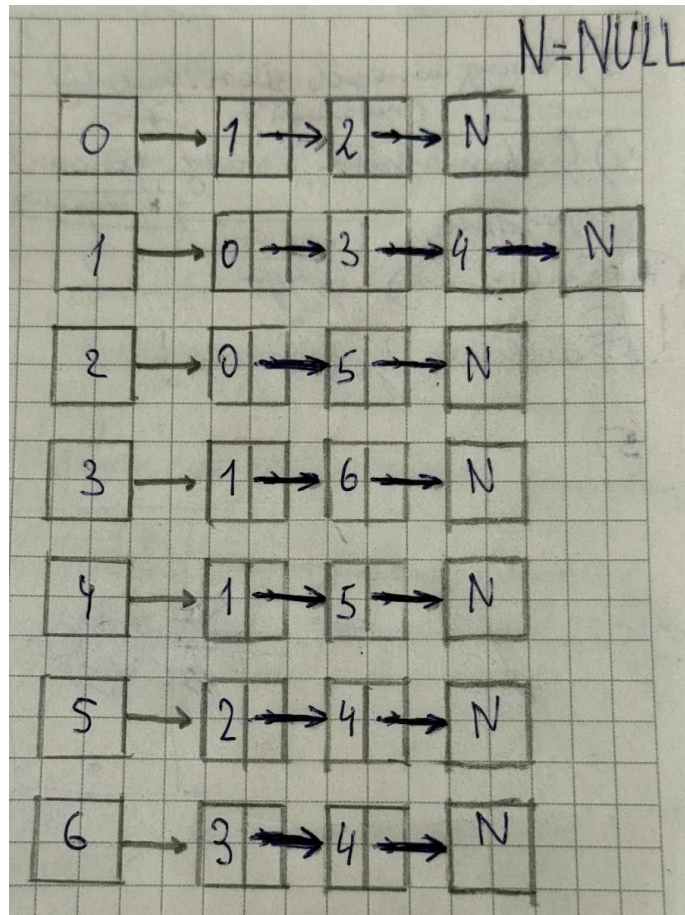
6x6

i \ j →	0	1	2	3	4
0	0	1	1	0	0
1	1	0	0	1	1
2	1	0	0	0	0
3	0	1	0	0	1
4	0	1	0	1	0

5x5

Матрица смежности $graph[i][j]$ хранит информацию о наличии ребра между вершинами i и j . Значение $graph[i][j] = 1$ означает наличие ребра, $graph[i][j] = 0$ - отсутствие ребра.

Структура хранения для представления графа списками смежных вершин:



Применяемые алгоритмы:

В обеих программах (для матрицы смежности и для списков смежности) реализован алгоритм поиска точек сочленения (articulation points) в неориентированном графе на основе поиска в глубину (DFS).

Текст программы:

Текст программы graph_matrix.cpp:

```

Текст программы:
Текст программы graph_matrix.cpp:
#include <iostream>
#include <fstream>
#include <windows.h>
using namespace std;
const int MAXN = 100;
int n;
int graph[MAXN][MAXN];
int disc[MAXN];
int low[MAXN];
int parent[MAXN];
bool visited[MAXN];
bool ap[MAXN];
int timer;

// функция для поиска точек сочленения
void dfs(int u)
{
    visited[u] = true;

```

```

        disc[u] = low[u] = ++timer;
        int children = 0;

        for (int v = 0; v < n; v++) // перебираем все вершины v
        {
            if (graph[u][v]) // если есть ребро между "u" и "v", тогда посещаем
непосещенных детей
            {
                if (!visited[v])
                {
                    children++; // увеличиваем счётчик детей; установка
родителя вершины v
                    parent[v] = u;
                    dfs(v); // рекурсивный вызов функции обхода для вершины
v

                    low[u] = min(low[u], low[v]); // Передаём наверх
минимальный disc из поддерева v

                    if (parent[u] == -1 && children > 1) // проверка есть ли у
корня более одного ребёнка
                        ap[u] = true;

                    if (parent[u] != -1 && low[v] >= disc[u]) // проверка
существует ли ребёнок v, из поддерева которого нет обратного ребра выше u
                        ap[u] = true;
                }
            }
            else if (v != parent[u]) // обратное ребро
            {
                low[u] = min(low[u], disc[v]);
            }
        }
    }
}

int main()
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    ifstream fin("graph_matrix.txt");
    if (!fin)
    {
        cerr << "Не удалось открыть файл graph_matrix.txt" << endl;
        return 1;
    }

    fin >> n;
    for (int i = 0; i < n; i++) // циклы для перебора строк и столбцов
    {
        for (int j = 0; j < n; j++)
        {
            fin >> graph[i][j]; // Заполняем матрицу данными из файла
        }
    }
    fin.close();

    // Вывод матрицы смежности
    cout << "Матрица смежности графа (" << n << " вершин):" << endl;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)

```

```

        cout << graph[i][j] << " ";
        cout << endl;
    }
    cout << endl;

    // вспомогательные массивы (инициализация)
    for (int i = 0; i < n; i++)
    {
        visited[i] = false;
        parent[i] = -1;
        ap[i] = false;
    }
    timer = 0; //счётчик времени входа в вершины

    // запускаем DFS для каждой компоненты связности
    for (int i = 0; i < n; i++)
    {
        if (!visited[i]) // начинаем поиск в глубину от вершины i
        {
            dfs(i);
        }
    }

    // вывод результатов
    cout << "Точки сочленения (цеха, отключение которых приведет к
проблемам (-_-) ):" << endl;
    bool found = false;
    for (int i = 0; i < n; i++)
    {
        if (ap[i])
        {
            cout << i << " ";
            found = true;
        }
    }
    if (!found)
    {
        cout << "Таких цехов нет.";
    }
    cout << endl;

    system("pause");
    return 0;
}

```

Текст программы graph_list.cpp:

```

#include <iostream>
#include <fstream>
#include <windows.h>
using namespace std;
const int MAXN = 100;

// Узел односвязного списка смежности
struct Node
{
    int data; // номер смежной вершины
    Node *next;
};

int n; // количество вершин
Node *adj[MAXN]; // массив указателей на списки смежности
int disc[MAXN];

```

```

int low[MAXN];
int parent[MAXN];
bool visited[MAXN];
bool ap[MAXN];
int timer;

// функция добавления ребра в список смежности (добавляем в начало)
void addEdge(int u, int v)
{
    Node *newNode = new Node; // выделяет память
    newNode->data = v; // записывает в узел номер смежной вершины v
    newNode->next = adj[u]; // новый узел вставляется в начало
    adj[u] = newNode; // голова списка вершины u теперь указывает на новый
узел
}

// Освобождение памяти, выделенной под списки
void deleteLists()
{
    for (int i = 0; i < n; i++)
    {
        Node *current = adj[i];
        while (current)
        {
            Node *temp = current;
            current = current->next;
            delete temp;
        }
        adj[i] = nullptr;
    }
}

// функция для поиска точек сочленения в списке смежности
void dfs(int u)
{
    visited[u] = true;
    disc[u] = low[u] = ++timer;
    int children = 0;

    // перебираем всех соседей вершины u по списку смежности
    Node *cur = adj[u];
    while (cur)
    {
        int v = cur->data; // номер соседней вершины
        if (!visited[v])
        {
            children++;
            parent[v] = u;
            dfs(v);

            low[u] = min(low[u], low[v]); // передаём наверх минимальный
disc из поддерева v

            // корень с более чем одним ребёнком
            if (parent[u] == -1 && children > 1)
                ap[u] = true;

            // некорневая вершина, из поддерева которой нет обратного ребра
выше u
            if (parent[u] != -1 && low[v] >= disc[u])
                ap[u] = true;
        }
        else if (v != parent[u]) // если v уже посещён и v не является
родителем u, то ребро (u,v) - обратное
    }
}

```

```

        {
            low[u] = min(low[u], disc[v]);
        }
        cur = cur->next;
    }
}

int main()
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    ifstream fin("graph_list.txt");
    if (!fin)
    {
        cerr << "Не удалось открыть файл graph_list.txt" << endl;
        return 1;
    }

    fin >> n;
    for (int i = 0; i < n; i++)
        adj[i] = nullptr; // обнуляем массив указателей adj

    // читаем списки смежности из файла
    for (int i = 0; i < n; i++)
    {
        int deg; // Для каждой вершины i читаем её степень deg (количество
смежных вершин)
        fin >> deg;
        for (int j = 0; j < deg; j++)
        {
            int v;
            fin >> v;
            addEdge(i, v); // Добавляем ребро в обоих направлениях (i -> v,
потом v -> i)
            addEdge(v, i);
        }
    }
    fin.close();

    // Вывод списков смежности
    cout << "Списки смежности графа (" << n << " вершин):" << endl;
    for (int i = 0; i < n; i++)
    {
        cout << i << ": ";
        Node* cur = adj[i];
        if (!cur) cout << "(нет соседей)";
        while (cur)
        {
            cout << cur->data << " ";
            cur = cur->next;
        }
        cout << endl;
    }
    cout << endl;

    // Инициализация вспомогательных массивов
    for (int i = 0; i < n; i++)
    {
        visited[i] = false;
        parent[i] = -1; // нет родителя (-1)
        ap[i] = false;
    }
    timer = 0;

```

```

        // запускаем функцию для каждой компоненты связности
        for (int i = 0; i < n; i++) // Цикл по всем вершинам. Если вершина ещё
        не посещена, запускает поиск в глубину
        {
            if (!visited[i])
                dfs(i);
        }

        // Вывод результатов
        cout << "Точки сочленения (цеха, отключение которых приведет к
        проблемам):" << endl;
        bool found = false;
        for (int i = 0; i < n; i++) // перебираем все вершины. Если ar[i]
        истинно, выводим номер вершины и ставим флаг found
        {
            if (ar[i])
            {
                cout << i << " ";
                found = true;
            }
        }
        if (!found)
            cout << "Таких цехов нет.";
        cout << endl;

        deleteLists();
        system("pause");
        return 0;
    }

```

Результаты работы программы:

При запуске программы (для графа 1) выводится матрица смежности и определяются точки сочленения, удаление которых приведет к нарушению целостности графа. Результат представлен на рисунке 1.

Матрица смежности графа (6 вершин):

```

0 1 1 0 0 0
1 0 0 1 0 0
1 0 0 0 1 1
0 1 0 0 0 0
0 0 1 0 0 1
0 0 1 0 1 0

```

Точки сочленения (цеха, отключение которых приведет к проблемам (--)):

```

0 1 2

```

Для продолжения нажмите любую клавишу . . . |

Рисунок 1

При запуске программы (для графа 2) выводится матрица смежности и определяются точки сочленения, удаление которых приведет к нарушению целостности графа. Результат представлен на рисунке 2.

```
Матрица смежности графа (5 вершин):
0 1 1 0 0
1 0 0 1 1
1 0 0 0 0
0 1 0 0 1
0 1 0 1 0

Точки сочленения (цеха, отключение которых приведет к проблемам (--)):
0 1
Для продолжения нажмите любую клавишу . . . |
```

Рисунок 2

При запуске программы (для графа 3) выводится список смежности и определяются точки сочленения, удаление которых приведет к нарушению целостности графа. Результат представлен на рисунке 3.

```
Списки смежности графа (7 вершин):
0: 2 2 1
1: 3 4 3 0
2: 2 5 0
3: 2 6 1
4: 3 6 5 1
5: 2 4 2
6: 2 4 3

Точки сочленения (цеха, отключение которых приведет к проблемам):
Таких цехов нет.
Для продолжения нажмите любую клавишу . . . |
```

Рисунок 3